Санкт-Петербургский государственный университет

РУБАНОВ Павел Евгеньевич

Выпускная квалификационная работа

Разработка системы для сегментации, контурирования и параметрического анализа биологических объектов

Уровень образования: бакалавриат

Направление *02.03.03 «Математическое обеспечение и администрирование информационных систем»*

Основная образовательная программа *СВ.5162.2020 «Технологии программирования»*

Научный руководитель:
зав. кафедры информационно-аналитических систем, к.ф-м.н. Михайлова Е. Г.

Рецензент:
м.н.с. лаборатории Гербарий, ООО «ГБС РАН», Шкурко А. В.

Санкт-Петербург
2026

Оглавление

Введение **4**

1. Постановка задачи **6**

1.1. Цель и задачи работы 6

2. Обзор существующих решений **8**

2.1. Инструменты для морфометрии и сегментации изображений 8

2.2. Сравнительный анализ 9

2.3. Выбор модели сегментации 10

3. Описание решения **14**

3.1. Требования к системе 14

3.2. Архитектура приложения 16

3.3. Конвейер обработки одного листа 19

3.4. Алгоритм параметрических измерений 20

3.5. Интерактивный режим сегментации 22

3.6. Пакетные режимы обработки 25

3.7. Полностью автоматический режим обработки 27

3.8. Отбор масок: классификатор листьев 28

3.9. Оптимизация производительности автоматической сегментации 34

3.10. Стек технологий 36

3.11. Тестирование 37

3.12. Сборка и публикация 38

3.13. Альтернативные подходы к отбору масок 38

4. Эксперимент **41**

4.1. Цель и методология 41

4.2. Метрики качества 41

4.3. Сравнение длин листьев 42

4.4. Сравнение поперечных профилей ширины 43

4.5. Итоговые показатели 43
4.6. Апробация 44

Заключение **45**

Список литературы **47**

Введение

Морфометрия — количественное описание формы биологических объектов — является одним из ключевых инструментов таксономии и экологии [1, 4]. Несмотря на развитый аппарат статистического анализа, получение исходных данных по-прежнему остаётся ручной или полуавтоматической операцией: исследователь вручную обводит контур каждого объекта в растровом редакторе и вручную снимает измерения. При сериях из десятков и сотен образцов это занимает значительное время и вносит субъективную погрешность, особенно когда измерения ведут несколько человек [2].

Задача осложняется при работе с морфологически сложными объектами. Листья мхов рода *Sphagnum* — экологически значимых эдификаторов торфяных болот и тундровых сообществ [3, 9] — состоят из одного слоя клеток, могут быть прозрачными и иметь неоднородные, рваные края. Существующие программы автоматической «обрисовки» контуров (например, tpsDIG [7]) требуют хорошо контрастированных объектов без инородных включений и не справляются с такими объектами.

С технической точки зрения проблема состоит в отсутствии инструмента, объединяющего в едином рабочем цикле нейросетевую сегментацию, интерактивную коррекцию результата и автоматическое извлечение морфометрических параметров с экспортом в табличный формат. Появление foundation-моделей сегментации — в частности, Segment Anything Model (SAM) [10] и её облегчённой версии MobileSAM [5] — открывает возможность строить такой инструмент без необходимости собирать размеченные обучающие данные. Настоящая работа направлена на решение описанной проблемы.

Работа имеет следующую структуру. В разделе 1 сформулированы цель, задачи и требования к разрабатываемой системе. В разделе 2 проведён обзор существующих инструментов и обоснован выбор модели сегментации. В разделе 3 описаны архитектура и реализация приложения. В разделе 4 приведены результаты сравнения с ручной разметкой и

апробации. Заключение содержит выводы и направления дальнейшего развития системы.

1. Постановка задачи

1.1. Цель и задачи работы

Работа выполняется в интересах лаборатории Гербарий ГБС РАН, сотрудники которой проводят морфометрический анализ листьев мохообразных (в первую очередь мхов рода *Sphagnum*).

Рабочий процесс биолога включает два принципиально разных этапа. Первый — *отбор образцов*: на каждом изображении присутствуют несколько листьев, и исследователь выбирает те из них, которые пригодны для измерений. «Хороший» лист — целый, с неповреждёнными краями, типичной для вида формы, расположен изолированно и не перекрывается с другими объектами. Листья с рваными или нечёткими краями, артефактами препарирования, атипичной морфологией или сильным перекрытием исключаются. Критерии отбора субъективны, определяются целями конкретного исследования и требуют экспертной оценки биолога; формализовать их в виде алгоритма невозможно. Второй этап — *сегментация и измерение* отобранных листьев — представляет собой технически однообразную ручную работу, которая при сериях из десятков и сотен образцов занимает значительное время и вносит субъективную погрешность.

Цель работы — разработать настольное приложение, автоматизирующее цикл «загрузка изображений → сегментация → ручная коррекция → параметрические измерения → экспорт», пригодное для использования без специальных вычислительных ресурсов на рабочих станциях биологической лаборатории.

Для достижения цели поставлены следующие **задачи**:

1. Проанализировать существующие программные решения для сегментации и морфометрии биологических объектов и обосновать выбор модели сегментации.

2. Сформулировать требования к системе совместно с заказчиком — сотрудниками лаборатории Гербарий ГБС РАН.

3. Спроектировать архитектуру приложения и реализовать его.

4. Реализовать алгоритм параметрических измерений (длина, поперечные профили ширины).

5. Сравнить результаты автоматической сегментации с ручной разметкой биологов и провести апробацию системы.

Практическая значимость работы состоит в сокращении времени, затрачиваемого на получение морфометрических данных, и в повышении воспроизводимости измерений за счёт стандартизованного алгоритма обработки.

2. Обзор существующих решений

2.1. Инструменты для морфометрии и сегментации изображений

Задача получения морфометрических данных с изображений биологических объектов решается несколькими классами инструментов.

ImageJ / FIJI [6] является стандартом де-факто в биологических лабораториях. Пакет предоставляет широкий набор инструментов для ручного обведения контуров и измерений, а также плагины для полуавтоматической пороговой сегментации (Threshold, Wand tool, Trainable Weka). Применительно к рассматриваемой задаче у FIJI выявлен ряд ограничений. Во-первых, пороговая сегментация требует ручной настройки порогов под каждую серию снимков и не справляется с прозрачными объектами и неоднородным фоном — типичными для препаратов листьев Sphagnum. Плагин Trainable Weka Segmentation позволяет использовать машинное обучение, однако требует ручной разметки обучающих областей для каждого нового типа объектов. Во-вторых, рабочий процесс в FIJI фрагментирован: сегментация, коррекция, измерения и экспорт выполняются разными инструментами, что требует экспертных знаний и увеличивает число ручных операций. В-третьих, FIJI не предоставляет встроенного инструмента для профильного анализа вдоль главной оси объекта с автоматическим определением направления оси — поперечные сечения исследователь расставляет вручную, что вносит субъективность в измерения.

tpsDIG2 [7] — специализированный инструмент для геометрической морфометрии, позволяющий расставлять landmarks и строить контуры (outlines). Требует хорошо контрастированных объектов без инородных включений и неоднородных краёв. Листья Sphagnum с рваными краями и прозрачной текстурой плохо поддаются автоматической «обрисовке».

PlantCV [11] — открытая Python-библиотека для анализа изображений растений. Поддерживает пакетную обработку и разнообразные

морфометрические функции (скелетизацию, измерение площади, периметра, длины). Не имеет графического интерфейса, требует написания скриптов, опирается на пороговые методы и морфологические операции без нейросетевой сегментации.

Labelme / CVAT / Label Studio — инструменты ручной разметки изображений, позволяющие строить полигональные аннотации. Предназначены для создания обучающих наборов данных, а не для морфометрии; не выполняют измерений.

2.2. Сравнительный анализ

Для сравнения выбраны критерии, непосредственно влияющие на трудоёмкость морфометрической обработки серий гербарных снимков.

Таблица 1: Сравнение существующих инструментов по ключевым критериям

Инструмент	ИИ-сегм. без обуч. данных	Работа на сложных объектах	Единый pipeline	Ручная коррекция	Профильный анализ по оси	Стандарт. вывод	Batch	Без GPU
ImageJ / FIJI	–	±	–	+	–	–	±	+
tpsDIG2	–	–	–	+	–	±	–	+
PlantCV	–	±	–	–	–	+	+	+
Labelme / CVAT	±	+	–	+	–	–	±	+
Разраб. система	+	+	+	+	+	+	+	+

+ — поддерживается; ± — частично / требует ручной настройки; – — не поддерживается

Сравнение показывает, что ни одно из рассмотренных решений не покрывает полный рабочий цикл биолога в рамках одного приложения. Наиболее близким аналогом является FIJI, однако его возможности применительно к задаче ограничены по нескольким ключевым направлениям.

Сегментация сложных объектов. Встроенные инструменты FIJI (пороговая сегментация, Wand) рассчитаны на контрастные объекты. Листья мохообразных с прозрачной клеточной структурой и рваными краями существенно снижают качество автоматической обрисовки, вынуждая пользователя выполнять значительный объём ручной правки.

Единый рабочий цикл. В FIJI сегментация, измерения и экспорт реализованы разными плагинами и требуют переключения между инстру-

ментами и ручного переноса данных. В разрабатываемой системе весь цикл — загрузка, сегментация, коррекция, измерения, экспорт — выполняется в одном окне приложения без переключения контекста.

Стандартизированный профильный анализ. FIJI позволяет измерять ширину листа, но направление оси и точки сечения задаются пользователем вручную для каждого образца. Разрабатываемая система автоматически определяет главную ось методом PCA и вычисляет ширину в фиксированных относительных точках $(0.125, 0.25, \ldots, 0.875)$, обеспечивая воспроизводимые и сопоставимые между собой профили для любой серии изображений.

2.3. Выбор модели сегментации

Ключевым техническим решением при разработке системы является выбор метода сегментации. Ниже рассмотрены три класса подходов и обоснован итоговый выбор.

Требование интерактивности и проблема отбора образцов

Прежде чем сравнивать подходы к сегментации, необходимо учесть ключевую особенность рабочего процесса биологов-морфометристов. На каждом изображении присутствует несколько листьев, и не все они пригодны для измерений. Биолог отбирает «хорошие» листья — целые, с неповреждёнными краями, типичной для вида формы, расположенные изолированно и не перекрывающиеся с другими объектами. Листья с рваными краями, артефактами препарирования или атипичной морфологией исключаются.

Этот отбор по своей природе субъективен: критерии зависят от целей конкретного исследования, вида и индивидуального опыта исследователя. Попытка формализовать «хорошесть» листа в виде числовых порогов (площадь, компактность контура, степень выпуклости) даёт лишь грубое приближение и требовала бы отдельной экспериментальной валидации на реальном материале — задача, выходящая за рамки настоящей работы. Таким образом, *полностью автоматический*

pipeline «обнаружить все листья → сегментировать → измерить» не отвечает требованиям заказчика: он не позволяет исключить непригодные образцы без участия эксперта.

Отсюда следует принципиальное требование к архитектуре системы: **эксперт должен участвовать в отборе**, а система — снять с него трудоёмкую часть работы. Идеальная модель сегментации в данном контексте — та, в которой *сам акт выбора листа и его сегментация совмещены в одно действие*: исследователь указывает на нужный лист, система немедленно строит маску.

Классические алгоритмы компьютерного зрения

Классические методы — пороговая сегментация, морфологические операции, алгоритм GrabCut, watershed — не требуют обучающих данных и работают на любом CPU. Однако они не удовлетворяют сформулированному выше требованию по двум причинам. Во-первых, их применимость ограничена качеством входного материала: листья Sphagnum могут быть практически прозрачными и иметь рваные края — именно те свойства, при которых пороговые методы дают фрагментированные маски с пропусками в области прозрачных клеток. Во-вторых, они не обеспечивают интерактивного отбора: алгоритм не знает, *какой именно* объект на изображении нужен биологу.

Специализированные нейросетевые модели (U-Net и аналоги)

Архитектуры типа U-Net [8] демонстрируют высокую точность сегментации биомедицинских изображений при наличии размеченного обучающего набора данных. В данном случае такого набора не существует: гербарная коллекция ГБС РАН содержит тысячи образцов нескольких видов рода *Sphagnum*, ни один из которых не был ранее сегментирован в машиночитаемом формате. Сбор репрезентативной обучающей выборки потребовал бы значительных трудозатрат самих биологов — той самой ручной работы, которую требуется автоматизировать. Помимо этого, обученная модель сегментирует все объекты подходящего

класса на изображении, не предоставляя механизма для интерактивного указания конкретного листа.

Foundation-модели сегментации: SAM и MobileSAM

Segment Anything Model (SAM) [10] — foundation-модель сегментации от Meta AI, обученная на датасете из более чем 1 млрд масок. Ключевое свойство SAM — способность сегментировать произвольный объект по интерактивным подсказкам (точки, ограничивающий прямоугольник) *без дообучения на конкретном домене.*

SAM органично решает проблему отбора образцов: один клик по телу листа одновременно является и выбором нужного объекта экспертом, и достаточной подсказкой для построения маски. Биолог просто не кликает на «плохие» листья — и они автоматически исключаются из анализа. Такой интерфейс не требует отдельного шага отбора и не накладывает на исследователя никакой дополнительной когнитивной нагрузки по сравнению с тем, что он и так делает мысленно.

Базовая версия SAM имеет существенное ограничение для настольного приложения: image encoder на основе ViT-H содержит 632 M параметров и требует GPU для приемлемой скорости работы.

MobileSAM [5] решает эту проблему: тяжёлый encoder заменён на компактный ViT-T (≈5.7 M параметров), обученный методом knowledge distillation на выходах оригинального SAM encoder. Размер checkpoint — 39 МБ против 2.6 ГБ у ViT-H SAM. Авторы показывают, что на задачах prompt-based сегментации качество MobileSAM сопоставимо с оригиналом [5], при этом на CPU среднего рабочего компьютера модель генерирует маску за 1–3 с — приемлемо для интерактивной работы.

Итог

MobileSAM выбран в качестве модели сегментации по совокупности факторов:

- не требует обучающих данных и обобщается на новые классы объектов;

- интерактивный prompt-based интерфейс естественно совмещает отбор образцов и сегментацию в одно действие;
- справляется с морфологически сложными объектами (прозрачные клетки, рваные края);
- работает на CPU, не требует GPU;
- умещается в дистрибутив настольного приложения (39 МБ).

SAM и MobileSAM не имеют собственного GUI и средств морфометрии — именно их интеграция в единый pipeline с интерактивной коррекцией и параметрическими измерениями и составляет основной вклад разрабатываемой системы.

3. Описание решения

3.1. Требования к системе

Требования сформулированы совместно с консультантом (м.н.с. лаборатории Гербарий ГБС РАН, Шкурко А. В.) на основе технического задания.

Требования заказчика

1. Автоматическое распознавание формы листа и построение полигональной маски контура.
2. Определение длины и ширины полигона.
3. Расстановка точек вдоль перпендикуляров к главной оси с заданным шагом (например, $\frac{1}{2}$ или $\frac{1}{4}$ длины листа).
4. Просмотр исходного изображения и маски рядом на экране.
5. Возможность задать масштабный коэффициент (1 пиксель = ... мм) для вывода измерений в физических единицах.
6. Экспорт результатов в табличный формат (CSV / Excel).
7. Пакетная обработка серий изображений.
8. Возможность ручной корректировки автоматически построенной маски.
9. Полностью автоматический режим обработки изображения: построение масок всех объектов без участия пользователя, автоматический отбор корректных листьев и запуск измерений.

Функциональные требования

На основе технического задания выделены следующие варианты использования:

- **UC1.** Загрузить набор изображений для обработки.
- **UC2.** Получить маску сегментации по интерактивным подсказкам (точки, ограничивающий прямоугольник).
- **UC3.** Скорректировать маску вручную в режиме кисти/ластика.
- **UC4.** Добавить подтверждённую маску в буфер или удалить её.
- **UC5.** Задать масштабный коэффициент пиксель → мм.
- **UC6.** Запустить пакетное измерение всех сохранённых масок.
- **UC7.** Экспортировать таблицу результатов в CSV / XLSX.
- **UC8.** Выбрать тип листа для автоматической обработки (из списка типов, для которых обучена модель отбора).
- **UC9.** Запустить полностью автоматический режим: система строит маски всех объектов на изображении, отбрасывает некорректные с помощью обученного классификатора и добавляет оставшиеся в буфер без участия пользователя.
- **UC10.** Просмотреть результаты автоматического отбора и при необходимости скорректировать отдельные маски вручную (переход в UC3).

Нефункциональные требования

- Работоспособность на стандартных рабочих станциях без выделенного GPU (инференс всех моделей на CPU). При наличии GPU приложение должно автоматически задействовать его для ускорения без дополнительной настройки.
- Время отклика модели сегментации не более 3 с на одно изображение в интерактивном режиме.
- Поддержка форматов JPEG, PNG, TIFF.

- Воспроизводимость результатов: одни и те же входные данные и подсказки дают одинаковый вывод.
- Единый формат хранения масок (PNG) и измерений (CSV / XLSX) для последующей статистической обработки.
- Расширяемость автоматического режима: добавление поддержки нового типа листа сводится к обучению классификатора отбора на соответствующей размеченной выборке без изменений в архитектуре приложения.

3.2. Архитектура приложения

Приложение реализовано на языке Python с использованием фреймворка PyQt6 для графического интерфейса и библиотеки PyTorch для инференса MobileSAM. Исходный код опубликован по адресу https://github.com/pavelrubanov/segmentation_system.

Кодовая база разделена на три независимых слоя (рис. 1):

- **Ядро (`src/core/`).** Вычислительные модули, не зависящие от UI: обёртка над MobileSAM (`predictor.py`), конвейер предобработки маски и канонизации листа (`leaf_pipeline.py`), модуль измерений (`leaf_measure.py`), универсальный конвейер пакетной обработки (`measurement_pipeline.py`), источники кропов (`sources.py`), экспорт CSV/XLSX (`export.py`), модель данных изображения (`image_da`
и конвенция имён файлов (`file_naming.py`).
- **Интерфейс (`src/ui/`).** Окна и виджеты PyQt6: навигационное окно (`navigation_window.py`), окно интерактивной сегментации (`segmentation/`), общие компоненты пакетных диалогов (`runner.py`), точки входа в пакетные режимы (`measure_crops.py`, `measure_images.py`
и тема оформления (`style.py`).
- **Классификатор (`classifier/`).** Намеренно изолированный пакет с собственными зависимостями: извлечение признаков замороженной MobileNetV3-Small (`mobilenet.py`), детерминированная

аугментация (`variants.py`), обучение XGBoost (`train.py`) и инференс (`predict.py`).

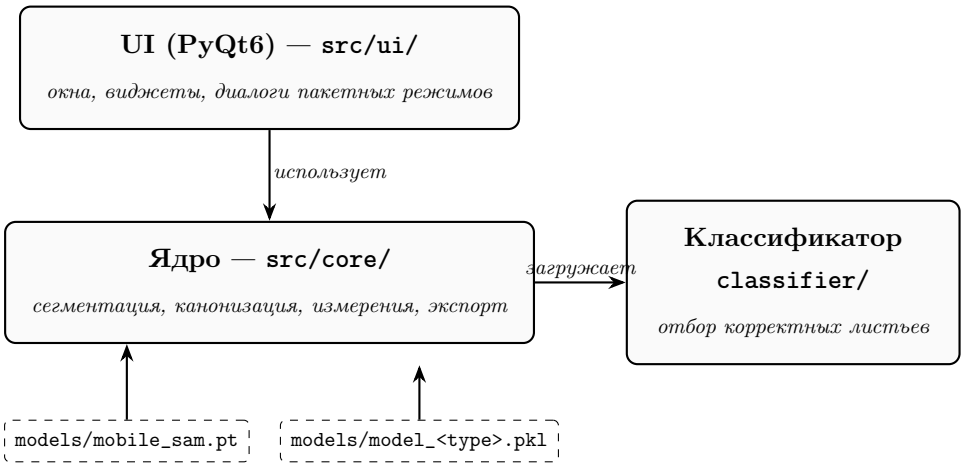

Рис. 1: Архитектура приложения: три независимых слоя и веса моделей, подгружаемые на старте

Точка входа — `src/main.py`. Скрипт принимает аргумент `--checkpoint` с путём к весам MobileSAM, создаёт единственный экземпляр `Predictor` (синглтон, который передаётся во все режимы работы) и запускает главное окно навигации (`NavigationWindow`, рис. 2). Из главного окна доступны три режима: интерактивная сегментация (`SegmentationWindow`), пакетные измерения по готовым кропам (`measure_crops`) и полностью автоматический режим (`measure_images`).

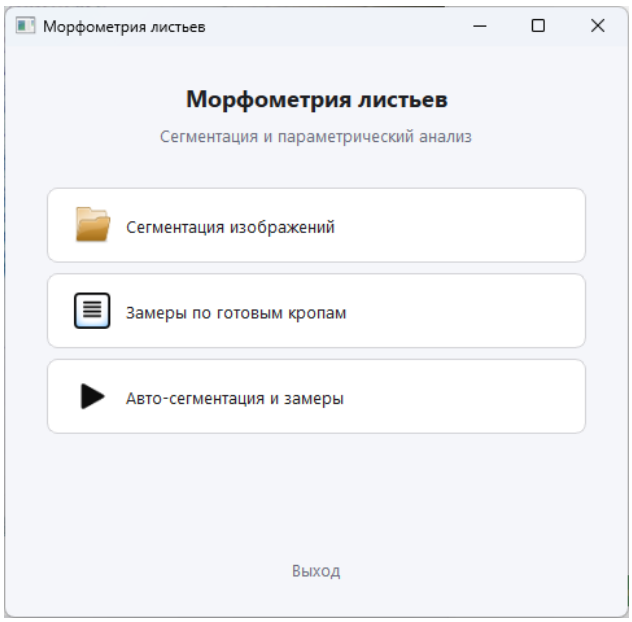

Рис. 2: Главное меню приложения — точка входа во все три режима работы

Конвенция имён файлов. Все три источника артефактов — ручная сегментация, авто-сегментация и пользовательский ввод — порождают и потребляют файлы по единому шаблону: `{stem}_leaf_crop_{idx:04d}` для кропа, `{stem}_mask_{idx:04d}.png` для маски и `{crop_stem}_vis.png` для визуализации измерений. Реализация и Qt-фильтры сосредоточены в модуле `file_naming.py`, что позволяет смешивать в одной папке результаты разных источников и переиспользовать их между режимами без отдельной конвертации.

Универсальный конвейер пакетной обработки. Оба пакетных режима (`measure_crops` и `measure_images`) используют один и тот же исполнитель `run_pipeline` из `measurement_pipeline.py`. Различие сводится к источнику кропов — объекту, удовлетворяющему типу `Callable[[str] Iterable[CropItem]]`. Для режима замеров по готовым кропам источником выступает функция `disk_source`, читающая один уже препроцессированный PNG; для авто-режима — класс `AutoSegmentSource`, для каждого исходного изображения вызывающий полный конвейер сегментации, фильтрующий результаты через классификатор и сохраняющий промежуточные артефакты на диск. Добавление нового сце-

нария пакетной обработки требует только реализации очередного источника кропов: воркер, диалог настроек, прогресс-бар и экспорт переиспользуются.

3.3. Конвейер обработки одного листа

Прежде чем измерить лист, его необходимо привести к каноническому виду. Сегментирующая модель возвращает бинарную маску, ориентированную произвольно на исходном изображении; после ручной коррекции маска может содержать мелкие артефакты (отдельные пиксели, дыры). Канонический вид — это RGB-фрагмент с обнулённым фоном, повёрнутый так, чтобы главная ось листа располагалась вертикально, а апекс находился сверху. Только в этой системе координат имеет смысл говорить о “длине” и “поперечной ширине” (рис. 3).

[Заглушка для рисунка: четыре панели — исходный фрагмент с маской,
очищенная маска, повёрнутый по PCA RGB-фрагмент,
итоговый кроп с обрезкой по bbox]

Рис. 3: Преобразование листа в конвейере: исходная маска → очистка → PCA-канонизация → кроп

Конвейер реализован в модуле `leaf_pipeline.py` и состоит из трёх шагов.

Шаг 1: очистка маски (`preprocess_mask`). В произвольной маске, полученной от MobileSAM или после ручного редактирования, могут оставаться побочные объекты и внутренние дыры. Реализация: поиск всех внешних контуров через `cv2.findContours`, выбор контура максимальной площади и заполнение получившейся области через `cv2.drawContours`

с флагом `FILLED`. На выходе — маска `uint8` (0 либо 255), содержащая ровно одну односвязную компоненту без дыр.

Шаг 2: канонизация ориентации (`pca_crop`). По координатам активных пикселей маски строится матрица ковариации 2×2, её собственные векторы вычисляются через `np.linalg.eigh`. Старший собственный вектор задаёт главную ось листа. Изображение с обнулённым фоном поворачивается аффинным преобразованием так, чтобы главная ось совпадала с вертикалью. После поворота определяется направление апекса: если центр масс пикселей лежит выше середины bbox, изображение переворачивается по вертикали — так апекс всегда оказывается сверху. Финальный шаг — обрезка по плотному bounding box активных пикселей; выходом является RGB-фрагмент, в котором лист занимает почти всю площадь и ориентирован вертикально.

Шаг 3: дедупликация набора масок (`dedupe_and_transform_leaf`). В автоматическом режиме MobileSAM AMG генерирует десятки кандидатов, среди которых встречаются дубликаты и вложенные маски. Маски сортируются по убыванию площади; для каждой следующей маски проверяется IoU с уже принятыми. Маска отбрасывается, если её IoU с любой принятой превышает порог 0,9. Для ускорения IoU вычисляется не на полном изображении: предварительно для каждой маски считается её ограничивающий прямоугольник, после чего пересечение масок выполняется только в области пересечения этих bbox, а объединение — по формуле $|A \cup B| = |A| + |B| - |A \cap B|$. К выжившим маскам применяется `pca_crop`; функция возвращает список пар (очищенная маска, канонический RGB-кроп).

3.4. Алгоритм параметрических измерений

Модуль `leaf_measure.py` принимает на вход *канонический* RGB-кроп после `pca_crop` (главная ось вертикальна, апекс сверху, фон чёрный). Эта конвенция позволяет отказаться от повторного построения контура и анализа главных компонент: все измерения сводятся к работе с маской активных пикселей в системе координат самого кропа

(рис. 4).

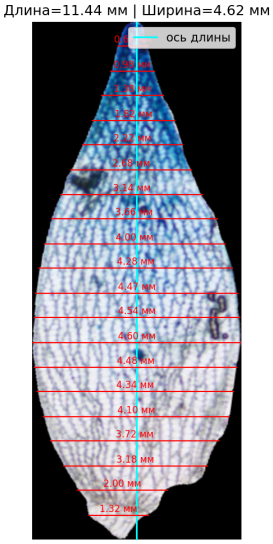

Рис. 4: Визуализация измерений одного листа: длина и семь поперечных сечений

Длина и максимальная ширина. Длина листа определяется как высота прямоугольника, ограничивающего активные пиксели кропа: $L = (y_{\max} - y_{\min} + 1) \cdot s$, где s — масштабный коэффициент (мм/пиксель), равный единице, если пользователь работает в пикселях. Максимальная ширина — ширина того же прямоугольника: $W = (x_{\max} - x_{\min} + 1) \cdot s$.

Поперечные сечения. Для каждой доли $f \in (0, 1)$ берётся строка $y = y_{\min} + \lfloor f \cdot (y_{\max} - y_{\min}) \rfloor$. В этой строке находятся крайний левый и крайний правый ненулевые пиксели x_l и x_r; ширина в сечении равна $w_f = (x_r - x_l) \cdot s$. По умолчанию используются семь сечений с долями 0,125; 0,25; 0,375; 0,5; 0,625; 0,75; 0,875, что соответствует равномерному разбиению длины листа на восемь интервалов; число сечений настраивается в диалоге пакетной обработки.

Результат. Метод возвращает структуру `LeafMetrics` с полями `length`, `width` и словарём `widths: dict[float, SectionWidth]` — для каждой доли хранятся сама ширина и координаты крайних точек, что используется при построении визуализации. Для каждого обработанного кропа дополнительно сохраняется изображение `*_vis.png` с нанесённой осью длины и поперечными сечениями. В режиме экспорта

в XLSX миниатюра этой визуализации встраивается в ячейку таблицы, что позволяет биологу визуально оценить корректность измерений непосредственно в Excel, не открывая файлы по отдельности.

3.5. Интерактивный режим сегментации

Интерактивный режим (окно `SegmentationWindow`) реализует основной сценарий работы биолога: эксперт самостоятельно выбирает листья на изображении, а система строит маску по подсказке и при необходимости даёт инструменты ручной коррекции (рис. 5). Типовой рабочий цикл показан на рис. 6.

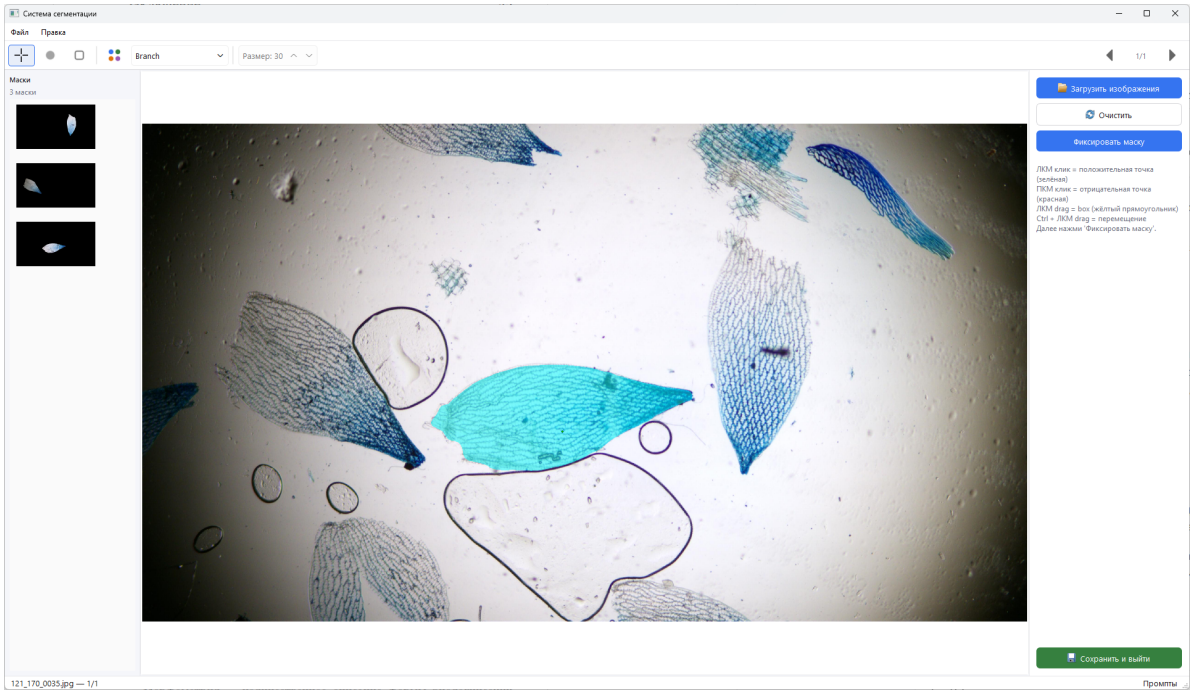

Рис. 5: Окно интерактивной сегментации

Холст и режимы работы. Центральный виджет — `SegmentationCanvas` на основе `QGraphicsView` с поддержкой зума к курсору и панорамирования (`Ctrl + ЛКМ`). Холст имеет два режима. В режиме *подсказок* клики левой и правой кнопкой мыши задают позитивные и негативные опорные точки, а перетаскивание ЛКМ — ограничивающий прямоугольник; после каждого изменения автоматически вызывается `Predictor.predict`, и над изображением отображается полупрозрачная маска. В режиме

редактирования ЛКМ работает кистью, а ПКМ — ластиком; диаметр кисти регулируется ползунком в тулбаре в диапазоне $1 \ldots 200$ пикселей. Маска хранится в виде ARGB-слоя (`QImage Format_ARGB32`); переключение между режимами не теряет промежуточный результат.

Буфер масок и асинхронная запись на диск. Подтверждённые маски попадают в виджет-список `MasksBuffer` (левая панель). Для каждой маски сохраняются два файла — собственно бинарная маска и RGB-кроп, прошедший канонизацию по `transform_leaf`; при выходе из окна они копируются в выбранную пользователем директорию. Запись PNG-файлов выполняется в пуле фоновых потоков (`ThreadPoolExecutor` с четырьмя воркерами), а в основном потоке хранятся только пути и уменьшенная копия исходного изображения (longest side 384 пикселя) для отрисовки иконок 128×128. Это позволяет работать с 4K-кадрами, не блокируя UI на запись больших PNG.

Кнопка авто-сегментации. В тулбаре доступна кнопка автоматической сегментации текущего изображения — она запускает функцию `auto_segment` (см. раздел 3.7) в отдельном `QThread`, после чего полученные маски наполняют буфер и отображаются на холсте. Это позволяет использовать авто-режим как стартовую точку для ручного режима: пользователь корректирует найденные системой маски, добавляет пропущенные и удаляет ложно-положительные.

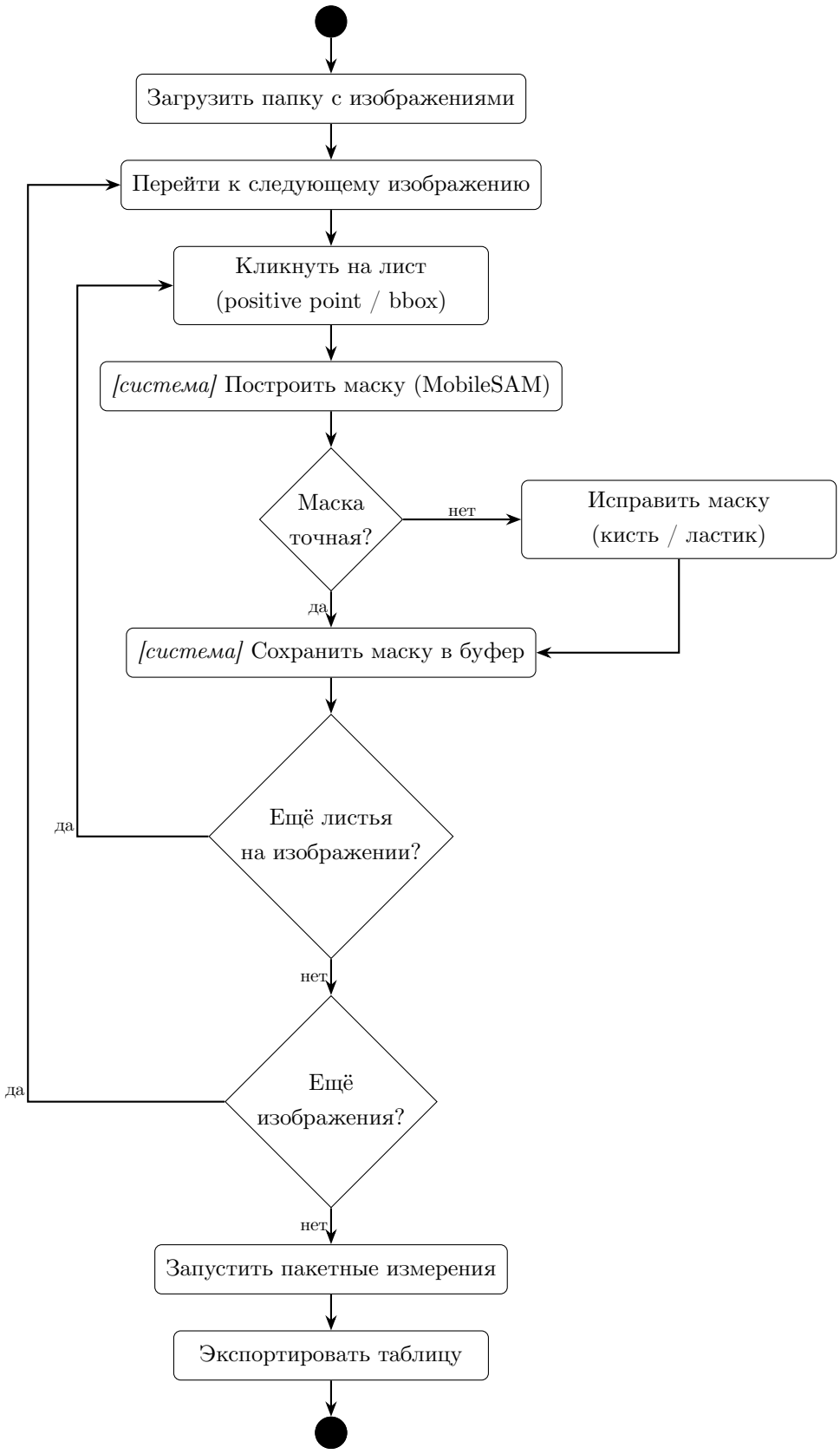

Рис. 6: Диаграмма активностей: рабочий цикл пользователя в интерактивном режиме

3.6. Пакетные режимы обработки

Помимо интерактивного режима приложение предоставляет два пакетных сценария, оба с единым диалогом настроек (рис. 7) и общим воркером `PipelineWorker`.

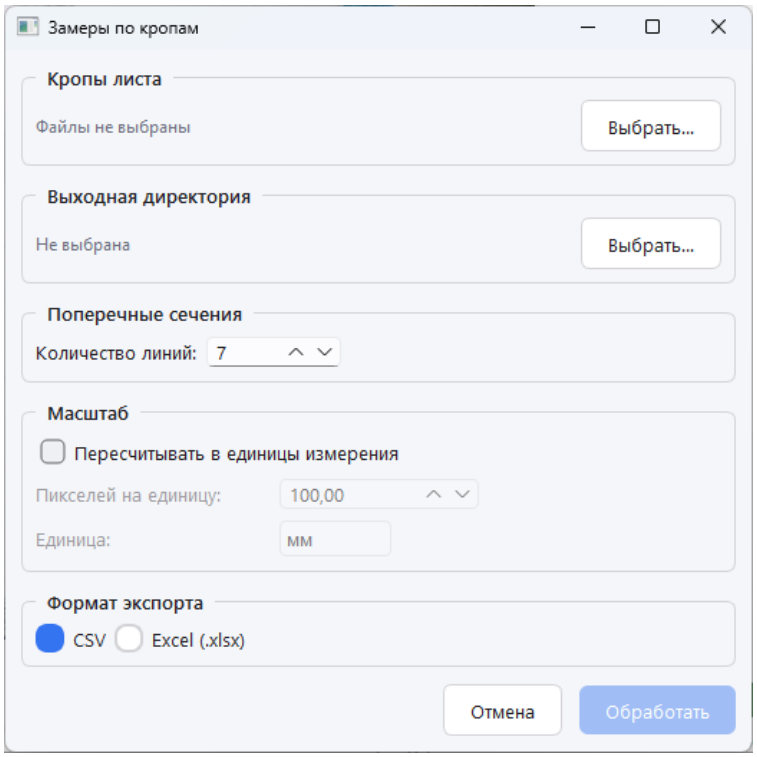

Рис. 7: Диалог настроек пакетной обработки

Замеры по готовым кропам (`measure_crops`). Сценарий используется, когда кропы уже подготовлены (вручную или ранее сохранены интерактивным режимом в формат `{stem}_leaf_crop_{idx}.png`). Источником кропов выступает `disk_source`: для каждого выбранного файла читается RGB-изображение и сразу подаётся в `measure_leaf`. По умолчанию формат экспорта — CSV, поскольку пользователю обычно требуется только сводная таблица.

Полностью автоматический режим (`measure_images`). В этом режиме на вход подаются исходные изображения, а сегментация и отбор корректных листьев выполняются без участия пользователя. Источник кропов — `AutoSegmentSource`: для каждого изображения запускается полный конвейер автоматической сегментации (раздел 3.7), по-

лученные кропы пропускаются через классификатор отбора (раздел 3.8), оставляются только классифицированные как `good_leaf`, после чего маски и кропы сохраняются в подпапки `masks/` и `crops/` выходной директории. Дополнительно в диалоге настроек присутствует выпадающий список типа листа — из него выбирается обученная модель отбора (`models/model_<type>.pkl`). Формат экспорта по умолчанию — XLSX, в столбце `crop` которого встроены миниатюры визуализаций измерений; такой отчёт позволяет биологу пройти по списку и быстро отбросить листья с заведомо неправильной сегментацией.

Типовой рабочий цикл пакетного режима показан на рис. 8. Воркер исполняет общий конвейер `run_pipeline`, периодически обновляя прогресс-бар; по завершении пользователь получает диалог с числом обработанных файлов, числом ошибок и путём к итоговой таблице. Ошибки на отдельных файлах не прерывают пакет — они накапливаются в результате и отображаются по окончании.

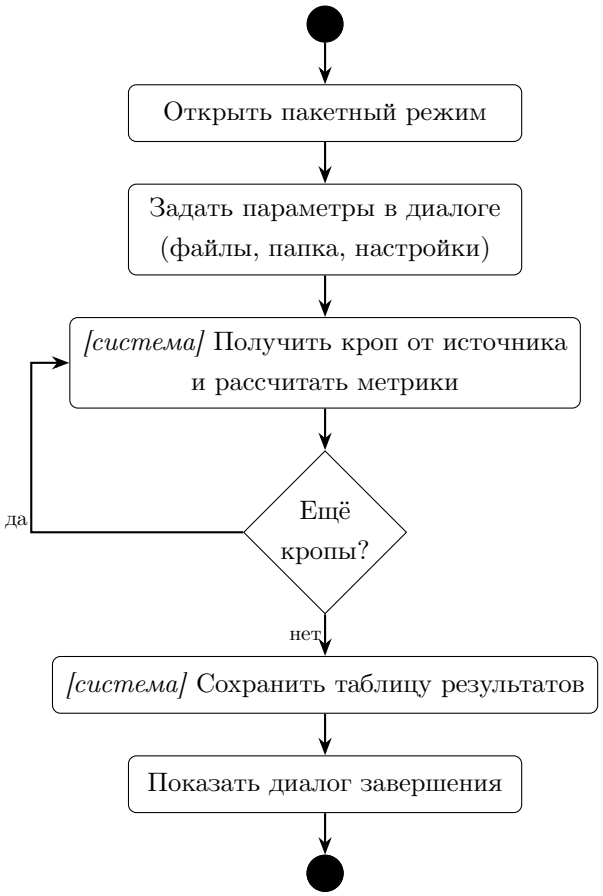

Рис. 8: Диаграмма активностей: рабочий цикл пакетной обработки

3.7. Полностью автоматический режим обработки

Полностью автоматическая сегментация реализована единой функцией (`auto_segment` в модуле `leaf_pipeline.py`), вызываемой как из интерактивного окна (по кнопке авто-сегментации), так и из пакетного режима. На вход подаётся RGB-изображение и инициализированный экземпляр `Predictor`, на выход — список пар (бинарная маска, канонический RGB-кроп) в координатах исходного изображения. Внутри объединяются четыре шага.

1. **Уменьшение масштаба.** Изображение уменьшается в два раза по каждому измерению методом area-resampling. Соответствующий коэффициент уменьшения вынесен в константу модуля; обоснование и выигрыш от этого шага рассмотрены в подразделе 3.9.

2. **Автоматическая сегментация.** На уменьшенном изображении запускается `SamAutomaticMaskGenerator` (метод `predict_all` обёртки `Predictor`) с параметрами `points_per_side` = 8, `points_per_batch` = 64, `pred_iou_thresh` = 0,80, `stability_score_thresh` = 0,88. Маски, площадь которых меньше 0,1 % или больше 70 % кадра, отфильтровываются.

3. **Очистка и дедупликация.** На том же уменьшенном масштабе выполняется конвейер очистки маски и канонизации листа, описанный в разделе 3.3; на выходе — список пар (очищенная HD-маска, канонический HD-кроп).

4. **Возврат к исходному разрешению.** Каждая выжившая маска апсэмплится к полному разрешению билинейной интерполяцией с последующей бинаризацией по порогу 127; канонический кроп масштабируется в два раза. Повторная PCA-канонизация на полном разрешении не выполняется — использование уже посчитанного HD-кропа обосновано в подразделе 3.9.

Результат работы автоматической сегментации, наложенный на исходное изображение, показан на рис. 9: каждая найденная маска отри-

сована своим цветом, видны как корректные одиночные листья, так и кандидаты с дефектами, которые далее отсеиваются классификатором.

Рис. 9: Результат автоматической сегментации в окне ручной разметки

В отличие от интерактивного режима, где эксперт верифицирует каждую маску визуально, в полностью автоматическом режиме ошибка сегментации автоматически попадает в выходную таблицу и влияет на результат исследования. Поэтому за этапом автоматической сегментации обязательно следует фильтрация по классификатору отбора (раздел 3.8): только маски, классифицированные как корректный одиночный лист, попадают в итоговый набор.

3.8. Отбор масок: классификатор листьев

В полностью автоматическом режиме каждая маска-кандидат, полученная на этапе автоматической сегментации, проходит через классификатор отбора: только объекты, классифицированные как корректные одиночные листья, попадают в итоговую таблицу измерений. Подзадача оформлена в виде независимого пакета `classifier/` и решается двухэтапной моделью — замороженный экстрактор признаков на осно-

ве свёрточной сети, предобученной на ImageNet, и градиентный бустинг XGBoost, обучаемый на этих признаках (рис. 10). Такой подход не требует дообучения свёрточной части под предметную область и устойчиво работает на сотнях обучающих примеров на тип листа.

[Заглушка для рисунка: схема конвейера классификатора — кроп листа → letterbox 512 × 512 → MobileNetV3-Small (frozen, ImageNet)
→ вектор 576 признаков → XGBoost (300 деревьев, 3 класса)
→ метка `good_leaf` / `bad_leaf` / `non_leaf`]

Рис. 10: Конвейер классификатора отбора масок

Извлечение признаков

Используется MobileNetV3-Small из `torchvision`, предобученная на ImageNet. Финальный классификационный слой заменяется на `Identity`, веса сети заморожены. Каждый кроп приводится к квадрату 512 × 512 пикселей с сохранением пропорций (letterbox — паддинг чёрным), нормализуется по статистикам ImageNet и пропускается через сеть; на выходе — вектор 576 признаков. Такой подход — использование сети, обученной на ImageNet, в режиме переноса знаний (transfer learning) — обоснован тем, что нижние и средние слои MobileNetV3 уже умеют выделять текстуру, цвет и базовые формы; этого достаточно для последующего разделения трёх грубых классов (хороший лист / плохой лист / не лист), и нет необходимости обучать свёрточную часть на небольшой размеченной выборке листьев. Извлечение признаков выполняется один раз на этапе сборки обучающего датасета (модуль `mobilenet.py`, CLI-команда `python -m classifier.mobilenet`); в инференсе те же призна-

ки вычисляются на лету.

Аугментация

Размеченная выборка для каждого типа листа сравнительно небольшая (от ~ 100 до ~ 600 примеров класса `good_leaf`), поэтому для каждого исходного кропа генерируются шесть детерминированных вариантов: горизонтальное отражение (два состояния) перемножается с тремя цветовыми трансформациями — оригинал, осветление (brightness = 1,2, contrast = 1,15, saturation = 0,9) и затемнение (0,8, 0,85, 1,1). Аугментация фиксирована в коде (модуль `variants.py`) и одинакова между запусками, что обеспечивает воспроизводимость экспериментов. Признаки MobileNetV3 вычисляются для каждого варианта отдельно и сохраняются в обучающий кэш `features.npz`, что устраняет необходимость пересчёта при повторных обучениях XGBoost.

Классификатор

XGBoost (`XGBClassifier`, `multi:softprob`) с гиперпараметрами: 300 деревьев, максимальная глубина 6, темп обучения 0,1, `subsample` = 0,8, `colsample_bytree` = 0,8, `tree_method` = `"hist"`. Веса классов обратно пропорциональны их численностям — это компенсирует существенный дисбаланс ($\sim 100-600$ позитивных против 1500 `bad_leaf` и ~ 1000 `non_leaf`). Модель обучается на трёх классах:

- `good_leaf` — корректный одиночный лист, пригодный для морфометрических измерений;
- `bad_leaf` — лист с дефектами (перекрытия с другими объектами, неполная форма, рваные края);
- `non_leaf` — объект нелистовой природы (фон, тень, артефакт сегментации).

На этапе фильтрации в `AutoSegmentSource` сохраняются только кропы класса `good_leaf`; разделение между `bad_leaf` и `non_leaf` имеет

диагностическое значение (позволяет в будущем по-разному обрабатывать эти два типа отказов), но в текущей реализации они одинаково отбрасываются.

Поддержка нескольких типов листьев *Sphagnum*

Морфология листьев у разных групп существенно различается — ветвевые листья имеют гладкий контур, стеблевые *capillifolium* отличаются вытянутой формой, *girgensohnii* — более широкой пластинкой. Один универсальный классификатор работает существенно хуже специализированных, поэтому для каждого типа листа обучается отдельная модель `models/model_<type>.pkl`: в качестве позитивного класса берутся "хорошие" листья из соответствующей подпапки `good_leaf_<type>`, негативные классы `bad_leaf` и `non_leaf` являются общими для всех типов и не требуют повторной разметки. В диалоге авто-режима пользователь выбирает тип листа — система автоматически подгружает соответствующие веса. Добавление поддержки нового типа сводится к разметке выборки `good_leaf_<type>` и запуску команды `python -m classifier.train`: код приложения при этом не меняется.

Процедура обучения

Использовалось hold-out разбиение 60/20/20 (train/val/test) с фиксированным `random_state`: train размножается шестикратно через аугментацию, val и test остаются без аугментации (только базовый вариант). На train обучается XGBoost с указанными гиперпараметрами; val используется для финального контроля (без раннего останова, поскольку число деревьев фиксировано), test — для независимой оценки качества. Дополнительно проведена 5-кратная стратифицированная кросс-валидация на полном датасете для оценки устойчивости качества к выбору разбиения.

Результаты

Распределение классов в обучающих выборках существенно несбалансированное: позитивный класс `good_leaf` составляет от 4,6 % (для `capillifolium`) до 18,5 % (для `branch`) объёма выборки (табл. 2). Поэтому *общая accuracy не является информативной метрикой* — тривиальный классификатор, всегда предсказывающий `bad_leaf`, получит accuracy около 0,55, не приняв при этом ни одного корректного листа. В качестве сводной оценки используется macro-F1 — невзвешенное среднее F1-меры по трём классам, дающее равный вес каждому из них независимо от поддержки. Все метрики приводятся в формате mean ± std, где среднее и выборочное стандартное отклонение вычислены по пяти фолдам стратифицированной кросс-валидации.

Таблица 2: Распределение классов в обучающей выборке (по типам листа)

Тип листа	`good_leaf`	`bad_leaf`	`non_leaf`	**Доля `good`**
`branch`	566	1500	994	18,5 %
`capillifolium`	121	1500	994	4,6 %
`girgensohnii`	309	1500	994	11,0 %

Сводные метрики приведены в табл. 3. Помимо macro-F1 указана F1-мера позитивного класса `good_leaf` — она наиболее важна с прикладной точки зрения, поскольку именно объекты этого класса попадают в итоговую таблицу измерений.

Таблица 3: Сводные метрики классификатора (5-кратная CV, mean ± std)

Тип листа	**Macro-F1**	**F1 (`good_leaf`)**
`branch`	0,944 ± 0,013	0,936 ± 0,022
`capillifolium`	0,940 ± 0,010	0,903 ± 0,024
`girgensohnii`	0,937 ± 0,005	0,913 ± 0,027

Подробные метрики по классам (precision, recall, F1), усреднённые по тем же пяти фолдам, приведены в табл. 4. Столбец **Support** показывает суммарную численность класса по всем пяти фолдам.

Таблица 4: Покомпонентные метрики классификатора (5-кратная кросс-валидация, mean $\pm$ std)

Тип листа	Класс	Precision	Recall	F1	Support
branch	good_leaf	$0{,}937 \pm 0{,}022$	$0{,}936 \pm 0{,}037$	$0{,}936 \pm 0{,}022$	566
	bad_leaf	$0{,}951 \pm 0{,}014$	$0{,}937 \pm 0{,}017$	$0{,}944 \pm 0{,}012$	1500
	non_leaf	$0{,}943 \pm 0{,}013$	$0{,}963 \pm 0{,}004$	$0{,}953 \pm 0{,}006$	994
capillifolium	good_leaf	$0{,}891 \pm 0{,}052$	$0{,}917 \pm 0{,}030$	$0{,}903 \pm 0{,}024$	121
	bad_leaf	$0{,}968 \pm 0{,}009$	$0{,}955 \pm 0{,}014$	$0{,}961 \pm 0{,}008$	1500
	non_leaf	$0{,}947 \pm 0{,}016$	$0{,}963 \pm 0{,}015$	$0{,}955 \pm 0{,}012$	994
girgensohnii	good_leaf	$0{,}900 \pm 0{,}042$	$0{,}926 \pm 0{,}019$	$0{,}913 \pm 0{,}027$	309
	bad_leaf	$0{,}957 \pm 0{,}008$	$0{,}940 \pm 0{,}009$	$0{,}948 \pm 0{,}005$	1500
	non_leaf	$0{,}943 \pm 0{,}015$	$0{,}959 \pm 0{,}018$	$0{,}951 \pm 0{,}014$	994

Анализ. Macro-F1 устойчиво превышает 0,93 для всех трёх типов листьев, причём стандартное отклонение по фолдам не превышает 1,3 % — качество подхода нечувствительно к выбору разбиения. Прикладная метрика F1 позитивного класса `good_leaf` находится в диапазоне $0{,}90-0{,}94$: не более $\sim 10\,\%$ корректных листьев теряются на этапе фильтрации, и не более $\sim 11\,\%$ проходящих фильтр кропов оказываются бракованными или нелистовыми объектами. Самая высокая дисперсия — у precision позитивного класса для `capillifolium` ($\pm 0{,}052$): прямое следствие меньшего объёма размеченных позитивных примеров (121 против 566 для `branch`). Расширение разметки этого типа — естественное направление дальнейшей работы. Класс `non_leaf` устойчиво отделяется на всех типах (F1 $\geq 0{,}95$), поскольку признаки ImageNet-сети хорошо разделяют биологические объекты и посторонний фон. Сопоставление автоматических измерений, полученных на отфильтрованных таким образом масках, с ручной разметкой эксперта приведено в разделе 4.

3.9. Оптимизация производительности автоматической сегментации

Прямолинейная реализация автоматической сегментации — запуск AMG на исходном 4K-кадре, очистка масок и сохранение PNG-файлов синхронно с UI — работала недопустимо медленно. На ноутбуке без GPU обработка одного кадра 3840×2160 занимала $\approx 18{,}7$ с, что для типового пакета в 50 снимков давало порядка пятнадцати минут. В интерактивном режиме UI зависал на ≈ 3 с при сохранении PNG-файлов. Это сделало необходимым целенаправленную оптимизацию конвейера.

Замеры выполнялись на ноутбуке (CPU, без GPU) на 4K-кадре с 13 листьями *Sphagnum* после дедупликации. Архитектурные шаги оптимизации и их вклад в итоговое ускорение приведены в табл. 5.

Таблица 5: Время одного кадра 4K в конвейере автоматической сегментации по шагам оптимизации

Версия	**SAM**	**dedupe**	**upscale**	**show**	**save**	**Total**
0. Исходная (4K)	6,30	8,54	—	0,77	2,89	**18,7**
1. Downscale 2× перед AMG	0,50	4,99	—	0,76	2,86	**9,2**
2. Единая `auto_segment` (dedupe HD)	0,50	1,17	2,89	0,77	2,84	**8,2**
3. Bbox-copy в апсэмпл-цикле	0,50	1,16	0,69	0,69	2,89	**5,9**
4. Skip 2-го `pca_crop`	0,50	1,17	0,10	0,69	2,90	**5,4**
5. Bbox-copy в `dedupe`	0,50	0,57	0,10	0,69	1,62	**3,5**
6. Async-запись PNG + `image_thumb`	0,30	0,57	0,10	0,71	0,12	**1,8**

Шаги обсуждаются в порядке применения.

Шаг 1. Downscale перед AMG. Энкодер SAM принудительно ресайзит вход к квадрату со стороной 1024 пикселей, поэтому стоимость энкодера от исходного разрешения не зависит. Однако пост-обработка масок (NMS, фильтрация, `stability_score`) масштабируется как $O(W \cdot H \cdot N)$ и для 4K оказывается в четыре раза дороже, чем для HD. После уменьшения входа в два раза время AMG сократилось с 6,30 с до 0,50 с. Маски апсэмплятся к исходному разрешению через `INTER_NEAREST`; потеря ~ 2 пикселей точности на границе не влияет на последующие измерения, поскольку дальнейшая обработка работает с целыми областями

маски.

Шаг 2. Единая точка входа автоматической сегментации. После шага 1 конвейер выглядел как метод `predict_all` обёртки `Predictor` (внутри — уменьшение, апсэмпл к исходному разрешению), за которым следовал дедуп и канонизация на полном разрешении. Дедуп выполнялся на масках, формально 4K, но фактически содержащих информацию HD-разрешения — то есть тяжёлые попарные операции IoU и обработка контуров делались на 8,3 Мрх без необходимости. Решение — вынести шаг уменьшения масштаба из обёртки `Predictor` в новую функцию (`auto_segment` в модуле `leaf_pipeline.py`), передавать в дедуп именно HD-маски, а апсэмпл к полному разрешению выполнять отдельным финальным циклом. Время этапа дедупа сократилось с 4,99 с до 1,17 с.

Шаг 3. Bbox-copy вместо `image.copy()` в апсэмпл-цикле. В финальном цикле для каждой маски выполнялось `masked = image.copy()` — полная копия 4K-кадра ($\approx$ 25 МБ), после чего вне маски пиксели обнулялись. На тринадцати листьях это давало $\approx$ 325 МБ копий и тяжёлую `warpAffine` на полном кадре. Замена: считать bbox каждой маски и копировать только соответствующий регион (`numpy`-срез возвращает view, копирование выполняется только при `.copy()`). Время апсэмпл-цикла: 2,89 $\rightarrow$ 0,69 с.

Шаг 4. Пропуск повторной PCA-канонизации. Дедуп уже выполняет PCA-канонизацию каждого выжившего листа на HD-кадре (внутри своего цикла); в финальном апсэмпл-цикле этот результат игнорировался, и канонизация запускалась повторно для полного разрешения. Один и тот же лист обрабатывался дважды. Замена: переиспользовать HD-кроп и масштабировать его к эквиваленту полного разрешения билинейной интерполяцией. Размытие на $2-3$ пикселя при таком масштабировании некритично — классификатор всё равно приводит кроп к 512×512, а измерения работают с целочисленными координатами bbox. Время апсэмпл-цикла: 0,69 $\rightarrow$ 0,10 с.

Шаг 5. Bbox-copy в дедупе. Тот же приём, что и в шаге 3, но применённый к финальному циклу внутри функции дедупа — там, где для каждой выжившей маски вычислялась PCA-канонизация. До замены

копировался полный HD-кадр ($\approx$ 6 МБ) на каждую маску — $\approx$ 78 МБ копий и аффинное преобразование на полном HD. После — работа только в bbox каждой маски. Время этапа дедупа: 1,17 $\rightarrow$ 0,57 с.

Шаг 6. Асинхронная запись PNG и кэш миниатюры изображения. В `MasksBuffer` запись каждой маски на диск выполнялась синхронно: копия полного 4K-кадра $\rightarrow$ обнуление пикселей вне маски $\rightarrow$ `Image.save` $\rightarrow$ загрузка PNG обратно для отрисовки иконки 128×128. На тринадцати масках это давало $\approx$ 2,9 с зависания UI. Решение состоит из трёх компонентов: (а) PNG-запись вынесена в `ThreadPoolExecutor` из четырёх потоков, futures хранятся в словаре `_pending` и при необходимости ожидаются точечно; (б) для отрисовки иконок один раз на изображение строится `image_thumb` — уменьшенная копия исходника с длинной стороной 384 пикселя ($\approx$ 250 КБ вместо 25 МБ), все иконки 128×128 строятся из неё; (в) операция “обнулить вне маски” заменена на `cv2.bitwise_and` — один SIMD-проход вместо двух. Время блока сохранения: 2,9 $\rightarrow$ 0,12 с, UI больше не зависает.

Итог. Совокупное ускорение — $\approx$ 10,4 раза (18,7 $\rightarrow$ 1,8 с на одном кадре). Для типового пакета в 50 снимков это означает сокращение времени обработки с пятнадцати до полутора минут и переводит автоматический режим из заведомо неприемлемого по времени в работоспособный для лабораторного цикла.

3.10. Стек технологий

Использованные технологии и обоснование выбора приведены в табл. 6.

Таблица 6: Использованные технологии

Компонент	**Технология**	**Обоснован**
Сегментирующая модель	MobileSAM (PyTorch)	Foundation m
Графический интерфейс	PyQt6	Кроссплатф
Обработка изображений	OpenCV, NumPy	Стандарт дл
Чтение изображений	Pillow	Поддержка
Извлечение признаков	MobileNetV3-Small	Лёгкая (5,4
Классификатор отбора	XGBoost	Высокое кач
Параллелизм при обучении	joblib	Многопроце
Визуализация измерений	matplotlib	Стандарт на
Экспорт CSV	csv (стандартная библиотека)	Совместимо
Экспорт XLSX с миниатюрами	openpyxl	Встраивание

3.11. Тестирование

Корректность модуля измерений верифицирована модульными тестами:

- *Прямоугольная маска* заданной ширины и высоты — ожидаемые длина и максимальная ширина совпадают с расчётными значениями с погрешностью менее 1 %;
- *Эллиптическая маска* — проверяется симметричность профиля поперечных сечений: ширины в долях f и $1-f$ должны совпадать;
- *Интеграционный тест* — загрузка реальных тестовых изображений, получение маски через `Predictor`, запуск `measure_leaf` и проверка формата CSV-таблицы.

Корректность конвейера автоматической сегментации и работы UI проверялась интеграционно — запуском приложения на тестовом наборе изображений и визуальной сверкой полученных масок и измерений с ручной разметкой эксперта (см. раздел 4).

3.12. Сборка и публикация

Целевая аудитория — сотрудники биологической лаборатории, для которых установка Python и его зависимостей не является штатной операцией. Доставка реализована в виде единого автономного дистрибутива под Windows 64 на базе PyInstaller 6.11.1 в режиме `--onedir` (конфигурация — `packaging/sphagnum.spec`, скрипт — `packaging/build_windows.ps` Один и тот же спек применим к CPU- и CUDA-вариантам `torch`.

Статический анализатор PyInstaller не видит динамических импортов `mobile_sam`, `timm` (модели регистрируются декораторами) и `xgboost` с `sklearn` (загружаются при `pickle.load` весов классификатора), поэтому эти пакеты подключаются явно через `collect_all`; для пакета `classifier` — `collect_submodules`. Тяжёлые неиспользуемые модули (`tkinter`, `IPython`, `jupyter`, `pytest`) исключаются из дистрибутива.

Отдельно потребовался runtime-хук для PyTorch: при первом запуске собранного приложения возникает `WinError 1114` на `c10.dll`, поскольку встроенный загрузчик `torch` использует строгий `LoadLibraryExW`, несовместимый с PyInstaller-frozen-контекстом. Хук (`packaging/runtime_hook` предзагружает критические DLL через `ctypes.CDLL` с более мягкими флагами; собственный цикл `torch` затем получает уже готовые хэндлы.

Итоговый дистрибутив `dist/sphagnum-cpu-win64/` занимает $\approx$ 920 МБ и распространяется в виде RAR-архива ($\approx$ 327 МБ). Запуск — двойным кликом по `sphagnum.exe`; консольное окно подавлено (`console=False`), стандартные потоки перенаправляются в `stdout.log` и `stderr.log` в директории дистрибутива.

3.13. Альтернативные подходы к отбору масок

Прежде чем остановиться на схеме MobileNetV3 + XGBoost (раздел 3.8), были опробованы два альтернативных подхода: гибридный классификатор с обучаемой свёрточной ветвью и одноклассовое обнаружение аномалий. Эксперименты проводились на ветвевых листьях (уровень 0, 778 кропов: 347 `good_leaf` + 280 `bad_leaf` + 151 `non_leaf`);

сводные метрики приведены в табл. 7.

Таблица 7: Сравнение альтернативных подходов к отбору масок (ветвевые листья, test)

Подход	**Модель**	**Accuracy**	**AUC**	**F1**	**Precision**	**Rec**
Supervised, 3 класса						
Гибридный CNN + MLP	полная модель	0,981	—	0,979	0,959	1,0
Гибридный CNN + MLP	MLP-only	0,853	—	—	—	–
One-class, бинарная задача "good vs аномалия"						
Автоэнкодер	per-pixel MSE	—	0,740	0,856	0,754	0,9
GMM (3 компоненты)	21-d признаки	—	0,992	0,979	0,986	0,9
OC-SVM (RBF)	21-d признаки	—	0,989	0,977	0,973	0,9

Гибридный классификатор с обучаемой свёрточной ветвью. Архитектура с двумя ветвями и поздним слиянием: лёгкая свёрточная сеть из трёх блоков (`Conv`–`BN`–`ReLU`–`MaxPool`, 3 $\to$ 128 каналов) обрабатывала RGB-фрагмент 96×96, параллельный MLP принимал 21-мерный вектор ручных признаков (статистики цвета RGB и HSV, инварианты Xy, эксцентриситет, плотность заполнения, solidity, circularity); выходные представления конкатенировались и подавались в общую классификационную голову $192 \to 64 \to 3$. Полная модель достигла точности 0,981 на test (156 примеров), отключение CNN-ветви снижало точность до 0,853 — то есть свёрточная часть действительно вносила вклад. Однако подход имеет два недостатка: во-первых, требует собственного обучения свёрточной части и расширения обучающей выборки при добавлении нового типа листа; во-вторых, ручные признаки требуют согласованности с конвейером препроцессинга, что усложняет сопровождение. Схема MobileNetV3 + XGBoost даёт сопоставимое качество без необходимости обучать свёрточную часть — замороженный экстрактор, обученный на миллионах изображений ImageNet, не уступает специально обучаемой лёгкой CNN на сотнях примеров.

Одноклассовое обнаружение аномалий. Рассматривались три модели, обучаемые исключительно на классе `good_leaf` и выдающие числовой score аномальности: свёрточный автоэнкодер с per-pixel MSE-оценкой, гауссова смесь (GMM) на 21-мерных ручных признаках и `OneClassSV`

с RBF-ядром на тех же признаках. GMM и OC-SVM показали высокое качество (AUC $\approx$ 0,99, F1 $\approx$ 0,98 на бинарной задаче), автоэнкодер — заметно ниже (AUC 0,740): ошибка реконструкции слабо разделяет хорошие листья и аномалии при работе с RGB-фрагментами на чёрном фоне. Несмотря на конкурентные метрики, подход отбракован по двум причинам. Во-первых, для подбора порога отсечения всё равно требуется размеченная аномальная выборка — формальное преимущество "обучение только на хороших примерах" на практике не реализуется. Во-вторых, схема не различает `bad_leaf` и `non_leaf`, тогда как разделение этих типов отказов сохраняет диагностическую ценность для последующего анализа ошибок. По совокупности факторов supervised-схема с тремя классами оказалась более удобной как с точки зрения интеграции в приложение, так и с точки зрения интерпретации результатов.

4. Эксперимент

4.1. Цель и методология

Целью эксперимента является количественная оценка точности автоматических морфометрических измерений системы путём сравнения с ручными измерениями, выполненными специалистом лаборатории Гербарий ГБС РАН.

Тестовая выборка включает 9 изображений листьев *Sphagnum* из гербарных коллекций. Для каждого изображения специалист выполнил ручное измерение двух морфометрических характеристик:

- длины листа по главной оси;
- поперечных профилей ширины в семи равноотстоящих сечениях (на долях 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875 от полной длины).

Те же изображения были обработаны разработанной системой с минимальным числом подсказок — как правило, один-два положительных клика по телу листа. Автоматические маски сохранены в директории `test/results/`; ручные эталонные маски — в `test/mapped/`.

4.2. Метрики качества

Для оценки погрешности измерений использовались следующие метрики:

$$\text{Bias} = \frac{1}{N}\sum_{i=1}^{N}(x_i^{\text{авт}} - x_i^{\text{руч}}), \qquad \text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|x_i^{\text{авт}} - x_i^{\text{руч}}|,$$

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i^{\text{авт}} - x_i^{\text{руч}})^2}, \qquad \text{MAPE} = \frac{1}{N}\sum_{i=1}^{N}\frac{|x_i^{\text{авт}} - x_i^{\text{руч}}|}{x_i^{\text{руч}}},$$

где $x_i^{\text{авт}}$ — автоматическое измерение, $x_i^{\text{руч}}$ — ручное измерение. Дополнительно вычислялся коэффициент корреляции Пирсона R между автоматическими и ручными значениями.

Положительный Bias означает систематическое завышение автоматических значений относительно ручных.

4.3. Сравнение длин листьев

В таблице 8 приведены автоматические и ручные измерения длины для всех девяти образцов. Все автоматические значения превышают ручные, что свидетельствует о систематическом завышении.

Таблица 8: Сравнение автоматических и ручных измерений длины листа (пиксели)

№	**Образец**	**Авто**	**Вручную**	Δ, px	APE, %
1	121_170_0037	1132.2	1106.2	+26.0	2.4
2	2954_0111	734.5	703.2	+31.3	4.4
3	2954_0115	1142.9	1132.0	+10.9	1.0
4	3013_0032	1547.2	1446.2	+101.0	7.0
5	3013_0040	2018.1	1992.2	+25.9	1.3
6	3275_0081	850.9	766.9	+84.0	11.0
7	3284_0007	1901.0	1680.4	+220.6	13.1
8	3726_0043	858.0	829.2	+28.8	3.5
9	l0105	443.8	408.2	+35.6	8.7
Итого				**+62.7**	**5.8**

Средняя абсолютная погрешность составила MAE = 62.7 px при RMSE = 88
и средней относительной погрешности MAPE = 5.8 %. Коэффициент корреляции $R = 0.994$ свидетельствует о высокой линейной согласованности автоматических и ручных измерений.

Наибольшие отклонения наблюдаются у образцов 3284_0007 и 3275_0081 (APE = 13.1 % и 11.0 %), что объясняется нестандартной формой листа: края листовой пластинки у этих образцов слабо выражены, и алгоритм PCA включает в проекцию на главную ось краевые пиксели маски, несколько удлиняя итоговое значение.

4.4. Сравнение поперечных профилей ширины

В таблице 9 представлены метрики погрешности для поперечных профилей по каждому из девяти образцов. Анализ проводился по всем 63 значениям (9 листьев $\times$ 7 сечений).

Таблица 9: Метрики погрешности поперечных профилей ширины (пиксели)

№	Образец	Bias	MAE	RMSE	MAPE, %
1	121_170_0037	−4.0	7.7	11.0	1.96
2	2954_0111	−4.1	9.4	11.4	1.80
3	2954_0115	−1.3	2.4	2.7	0.62
4	3013_0032	−12.7	14.1	17.1	1.58
5	3013_0040	−14.2	14.3	20.8	2.38
6	3275_0081	+7.3	13.6	16.1	3.93
7	3284_0007	+12.4	21.0	25.4	2.49
8	3726_0043	+0.1	3.3	4.0	0.78
9	l0105	−3.4	4.3	7.1	1.42
Итого		**−2.2**	**10.0**	**14.7**	**1.89**

Суммарная MAPE по поперечным профилям составила **1.89 %** — существенно ниже, чем по длинам. Коэффициент корреляции $R = 0.998$ при 63 парах наблюдений подтверждает высокую согласованность алгоритма сканирования поперечных сечений с ручными измерениями.

Систематический сдвиг по профилям незначителен ($\text{Bias} = -2.2\,\text{px}$) и не имеет выраженного направления: у одних образцов алгоритм слегка занижает ширину (листья 1, 2, 4, 5), у других — слегка завышает (листья 6, 7), что характерно для случайной погрешности краевого пикселя маски, а не для систематической ошибки метода.

4.5. Итоговые показатели

В таблице 10 сведены итоговые метрики по двум типам измерений.

Таблица 10: Итоговые метрики точности автоматических измерений

Тип измерения	N	**Bias, px**	**MAE, px**	**RMSE, px**	**MA**
Длина листа	9	+62.7	62.7	88.5	
Поперечные профили ширины	63	−2.2	10.0	14.7	

Измерение поперечных профилей демонстрирует высокую точность — погрешность менее 2 % приемлема для сравнительного морфометрического анализа. Длина листа имеет систематическое завышение порядка 5–6 %, обусловленное особенностью PCA-алгоритма: главная ось ориентируется по облаку пикселей маски и при наличии широкого основания или артефактов краевых пикселей несколько удлиняется. Данная погрешность стабильна и может быть скорректирована калибровочным коэффициентом.

4.6. Апробация

Прототип системы был продемонстрирован консультанту — м.н.с. лаборатории Гербарий ГБС РАН Шкурко А. В. По итогам демонстрации качество автоматической сегментации признано удовлетворительным для дальнейшего развития системы. Видеозапись демонстрации прилагается к работе.

В ходе апробации были выявлены следующие направления доработки:

- поддержка масштабного коэффициента (пересчёт пикселей в мм);
- возможность экспорта визуализаций в дополнение к CSV;
- расстановка точек вдоль периметра с заданным шагом (требование из технического задания).

Заключение

В ходе работы была разработана система для сегментации, контурирования и параметрического анализа биологических объектов по изображениям. Система реализована в виде кроссплатформенного настольного приложения на Python (PyQt6) и не требует GPU для работы.

Получены следующие результаты:

1. Проведён анализ предметной области и существующих инструментов. Показано, что ни одно из рассмотренных решений (ImageJ, tpsDIG2, PlantCV) не покрывает полный рабочий цикл биолога в едином приложении.

2. Сформулированы функциональные и нефункциональные требования совместно с консультантом из лаборатории Гербарий ГБС РАН.

3. Разработана архитектура системы, включающая модуль сегментации на основе MobileSAM, интерактивный холст с режимами подсказок и ручной коррекции, буфер масок и модуль пакетных измерений.

4. Реализован алгоритм параметрических измерений на основе PCA, вычисляющий длину и поперечные профили ширины листа в семи точках вдоль главной оси.

5. Проведено сравнение автоматической сегментации с ручной разметкой биологов на выборке из 9 листьев.

6. Система апробирована консультантом; качество сегментации признано удовлетворительным.

Направления дальнейшего развития:

- реализация масштабного коэффициента для вывода измерений в миллиметрах;

- расстановка точек вдоль периметра с заданным шагом;
- дообучение MobileSAM на специфических снимках Sphagnum для повышения точности;
- расширение тестовой выборки и проведение количественной апробации с биологами.

Список литературы

[1] Bookstein Fred L. Morphometric Tools for Landmark Data: Geometry and Biology. — Cambridge : Cambridge University Press, 1991.

[2] Bookstein Fred L. A Hundred Years of Morphometrics // Acta Zoologica Academiae Scientiarum Hungaricae. — 1998. — Vol. 44. — P. 7–59.

[3] Clymo R. S., Hayward P. M. The Ecology of Sphagnum // Bryophyte Ecology / Ed. by A. J. E. Smith. — Dordrecht : Springer, 1982. — P. 229–289.

[4] Dryden Ian L., Mardia Kanti V. Statistical Shape Analysis. — New York : John Wiley & Sons, 1998.

[5] Faster Segment Anything: Towards Lightweight SAM for Mobile Applications / Chaoning Zhang, Dongshen Han, Yu Qiao et al. // arXiv preprint arXiv:2306.14289. — 2023. — URL: https://arxiv.org/abs/2306.14289.

[6] Fiji: an open-source platform for biological-image analysis / Johannes Schindelin, Ignacio Arganda-Carreras, Erwin Frise et al. // Nature Methods. — 2012. — Vol. 9, no. 7. — P. 676–682.

[7] Rohlf F. J. tpsDIG2, version 2.16 [software]. — New York: Stony Brook University, Department of Ecology and Evolution. — 2021.

[8] Ronneberger Olaf, Fischer Philipp, Brox Thomas. U-Net: Convolutional Networks for Biomedical Image Segmentation // Medical Image Computing and Computer-Assisted Intervention (MICCAI). — 2015. — P. 234–241.

[9] Rydin Håkan, Gunnarsson Ulf, Sundberg Sebastian. The role of Sphagnum in peatland development and persistence // Boreal Peatland Ecosystems / Ed. by R. K. Wieder, D. H. Vitt. — Heidelberg : Springer, 2006. — Vol. 188 of Ecological Studies. — P. 47–65.

[10] Segment Anything / Alexander Kirillov, Eric Mintun, Nikhila Ravi et al. // Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). — 2023. — P. 4015–4026.

[11] A versatile phenotyping system and analytics platform reveals diverse temporal responses to water availability in Setaria / Noah Fahlgren, Malia Feldman, Malia A. Gehan et al. // Molecular Plant. — 2015. — Vol. 8, no. 10. — P. 1520–1535.